

Caliber

Everything an AI assistant (or a new developer) needs to work on this codebase: architecture, conventions, gotchas, and how to add features without breaking things.

Repo: github.com/vrajdesai17/Open-Atlas---AI-for-Social-Good-Hackathon-2026 · **Stack:** Next.js 16 · React 19 · TypeScript · Tailwind v4 · Groq · Postgres · **Use:** paste into Claude as context before editing.

01 What this project is

Caliber is a web app that assesses whether someone qualifies for a US "extraordinary ability" visa (O-1A, EB-1A, and siblings), transparently and for free.

A user pastes their public record (GitHub, publications, press links, résumé). The app fetches real data, an LLM extracts structured evidence, an **adversarial panel** judges each legal criterion (Advocate vs Examiner vs Adjudicator), and deterministic code applies the "meet N of M criteria" rule to produce a verdict. It then generates a gap plan, a draft petition letter, and a downloadable PDF. It also has a Policy Watch feed (recent immigration rule changes) and an F-1/H-1B pathway tool that bridges into the assessment engine.

The core principle, memorize it: AI proposes, code disposes. The LLM extracts and argues; deterministic code holds the legal rules, does all counting/math, and renders the final verdict. Never let the LLM decide the yes/no outcome or do arithmetic — that lives in code. This is what makes the tool accurate enough to trust.

02 Current status — what's built

AREA	STATE
Google OAuth sign-in (Auth.js v5) + dev quick-login	Done
Visa selector + per-visa assessment (O-1A, EB-1A, O-1B, EB-1B, EB-2 NIW)	Done
Data fetchers: GitHub, OpenAlex, résumé PDF parse	Done
Extractor agent (raw data → structured evidence)	Done
Adversarial panel + deterministic verdict (calibrated strict)	Done
Gap runway, draft petition letter, PDF export	Done
Policy Watch (Federal Register feed, AI-classified)	Done
F-1/H-1B pathway (odds, STEM-OPT, bridge)	Done
Optional DB-backed profile + first-login onboarding + prefill	Done
Saved assessment history; personalized Policy Watch alerts	Not built (DB ready)
Fully-independent panel agent calls; O-1B/NIW calibration; BLS wages	Not built

03 Tech stack

Framework	Next.js 16 (App Router, Turbopack) + TypeScript. React 19.
Styling	Tailwind CSS v4. Brand tokens defined with <code>@theme</code> in <code>app/globals.css</code> (e.g. <code>bg-accent</code> , <code>text-muted</code> , <code>bg-green-soft</code>). Light theme only.
Auth	Auth.js / NextAuth v5 beta. Google provider + a dev-only Credentials provider. JWT sessions, no DB adapter.
LLM	Groq, model <code>llama-3.3-70b-versatile</code> , via its OpenAI-compatible REST endpoint with JSON mode. All calls go through <code>lib/agents/llm.ts</code> .
Database	Postgres (Neon/Supabase) via <code>postgres.js</code> . Only used for profiles so far.
PDF	Browser print-to-PDF (<code>window.print()</code> + a <code>.print-report</code> block in <code>globals.css</code>). No PDF library.
Other deps	<code>unpdf</code> (résumé text extraction), <code>@google/genai</code> (only its <code>Type</code> enum is used to shape schemas — the actual calls are Groq).

Note on `@google/genai` : Gemini was the original LLM but its free tier caps new keys at ~20 requests/day, so the project moved to Groq. The package lingers only because `extractor.ts` / `panel.ts` import `Type` to build documentary schemas. You can ignore Gemini entirely.

04 How it works — the pipeline

```
sign in (Google/dev)
→ pick a visa          /assess/new → /assess/new/[visa]
→ ingest sources       POST /api/fetch-sources (GitHub + OpenAlex + résumé parse)
→ extract evidence     POST /api/extract (LLM: raw data → structured items)
→ adversarial panel    POST /api/panel (LLM: judge each criterion)
→ deterministic verdict (code counts "met", applies N-of-M threshold)
→ scorecard + gap runway (runway comes from the panel call)
→ draft letter         POST /api/letter (LLM, on demand)
→ PDF                  window.print() of a hidden .print-report region
```

The client component `app/assess/new/[visa]/AssessmentForm.tsx` orchestrates this whole sequence with React state (evidence → panel → letter), calling the API routes in turn. Each API route is auth-gated and thin; the real logic lives in `lib/`.

05 The adversarial panel (read before touching it)

For each visa criterion the panel produces three role outputs:

- **Advocate** — strongest good-faith argument the evidence meets the criterion.
- **Examiner** — how a skeptical USCIS officer would push back (the likely RFE).
- **Adjudicator** — verdict (`met` / `partial` / `gap`) + confidence + reasoning + runway.

Important implementation fact: the panel is ONE Groq call that returns an array of per-criterion verdicts (all three roles inside each), NOT three independent calls, and NOT one call per criterion. This is deliberate — Groq's free tier is ~1,000 req/day and 12k tokens/min, so per-criterion or per-role calls blow the limit. Keep it a single call. Fully-independent role calls (on borderline criteria only) is a documented future enhancement.

Calibration. `lib/agents/panel.ts` contains a "STANDARD OF REVIEW" block that makes the model grade like a skeptical officer (rejects student awards, low-tier journals, ordinary roles, un-benchmarked salaries). This calibration is tuned for the O-1A/EB-1A extraordinary-ability standard. If you improve O-1B (arts) or EB-2 NIW (the 3-prong Dhanasar test), add their own calibration — the current one is extraordinary-ability-shaped.

The verdict is code, not the model. `runPanel()` counts `verdict === "met"` and compares to `visa.threshold`. Do not move this into the prompt.

06 Repository map

auth.ts	Auth.js config (Google + dev provider), exports
auth/handlers/signIn/signOut	
app/	
layout.tsx	page.tsx
globals.css	Tailwind + @theme brand tokens + print styles
signin/page.tsx	sign-in (server actions call signIn)
assess/page.tsx	dashboard; redirects first-time users to onboarding
assess/new/page.tsx	visa selector
assess/new/[visa]/page.tsx	input page (server; validates visa)
assess/new/[visa]/AssessmentForm.tsx	MAIN client flow (ingest→extract→panel→letter→PDF)
assess/new/[visa]/Scorecard.tsx	verdict + expandable criterion cards
assess/new/[visa]/ReportPrint.tsx	hidden print-only report (the PDF)
policy-watch/page.tsx	Policy Watch feed
pathway/page.tsx	PathwayTools.tsx
profile/page.tsx	ProfileForm.tsx
api/*/route.ts	fetch-sources, extract, panel, letter, profile, auth
components/AppHeader.tsx	shared header + nav
lib/	
visas.ts	VISA rule packs (criteria + thresholds) – the deterministic skeleton
agents/llm.ts	Groq client: generateJSON(prompt, schema, opts)
agents/extractor.ts	raw data → structured evidence items
agents/panel.ts	adversarial panel + deterministic verdict
agents/letter.ts	draft petition letter
agents/policyClassifier.ts	summarize/classify Federal Register items
sources/{github,openalex,resume}.ts	data fetchers
policy/{federalRegister,watch}.ts	rule-change feed (+ 6h in-memory cache)
pathway.ts	H-1B odds + STEM-OPT (pure, deterministic)
db.ts	profile.ts
Postgres client + profile store	
scripts/test-*.mts	standalone smoke tests: npx tsx scripts/test-X.mts

07 Conventions & patterns

LLM calls

Always go through `generateJSON(prompt, schema, { temperature })` in `lib/agents/llm.ts`. It uses Groq JSON mode, retries transient errors + Groq's `json_validate_failed`, and returns parsed JSON. The `schema` arg is documentary — Groq does not enforce it, so **the prompt MUST explicitly state the exact JSON shape** ("Return a JSON object of exactly this shape: ..."). Always add guardrails after parsing (drop invalid enum values / invalid ids). Never trust the model for counts or the final decision.

API routes

```
export const runtime = "nodejs";
export async function POST(req) {
  const session = await auth();
  if (!session) return NextResponse.json({error:"unauthorized"}, {status:401});
  try { /* call lib fn */ return NextResponse.json(result); }
  catch (err) { return NextResponse.json({error: msg}, {status:500}); }
}
```

Auth in pages

Server components: `const session = await auth(); if (!session) redirect("/signin")`. Sign-in/out are server actions calling `signIn()/signOut()` from `@/auth`. There is no client `SessionProvider`.

Styling

Use the brand tokens (`accent`, `ink`, `muted`, `faint`, `line`, `panel`, `green/green-soft`, `amber/amber-soft`, `paper`). Cards: `rounded-2xl border border-line bg-card p-6`. Match the existing look.

The visa config pattern

Everything visa-specific is data in `lib/visas.ts` (`id, name, fullName, threshold, criteria[]`). The selector, input sidebar, panel, and scorecard all read from it. Add a visa = add a config object; no new components needed.

08 Environment & local setup

```
npm install
cp .env.example .env.local      # then fill in
npm run dev                     # http://localhost:3000 (or 3001 if 3000 is taken)
```

AUTH_SECRET	openssl rand -base64 32
AUTH_TRUST_HOST	true
AUTH_GOOGLE_ID / _SECRET	Google Cloud Console → OAuth client. Redirect URI must match your dev port: http://localhost:PORT/api/auth/callback/google .
GROQ_API_KEY	console.groq.com/keys (free)
DATABASE_URL	Neon or Supabase Postgres URL (?sslmode=require). Schema auto-creates on first use.

Each teammate needs their own keys in their own `.env.local` (it is gitignored — never commit it). Google OAuth: add each dev's own client, or share one and register every redirect port. In dev you can also use the "Continue as demo user" button, which needs no Google setup.

09 Gotchas & hard-won lessons

- **Groq rate limits are the main constraint:** ~1,000 requests/day, ~12,000 tokens/minute (free). Budget ~2–3 LLM calls per assessment. Do not add per-criterion or per-item LLM calls — batch into one structured call (see the panel).
- **Groq JSON mode needs the shape in the prompt** and occasionally returns `json_validate_failed` (400) — `llm.ts` already retries it. Keep batch outputs modest (Policy Watch classifies 8 items, not 20).
- **Dev port drift:** 3000 is often occupied, so Next runs on 3001. OAuth redirect URLs must match whatever port you use.
- **Editing `.env.local` carefully:** a value pasted with no trailing newline can get shuffled onto the next line during edits — check keys land on the right lines.
- **Never commit secrets.** `.env*` is gitignored; `.env.example` (placeholders) is force-added.
- **Model names move.** Groq/Gemini model IDs change over time; the model is a single constant in `lib/agents/llm.ts` (`MODEL`, override via `GROQ_MODEL`).
- **Honesty is a feature.** The panel is calibrated to be strict; the letter is honest when not eligible; Policy Watch never fabricates items. The judges are immigration lawyers — do not add over-claiming behavior.
- **LinkedIn has no usable API** — it was intentionally dropped. Don't reintroduce a scraper.

10 Recipes — how to add things

Add a new visa

Add an object to `VISAS` in `lib/visas.ts` with `id`, `name`, `fullName`, `category`, `tagline`, `threshold`, `criteria[]`. It automatically appears in the selector and works through the whole pipeline. For non-extraordinary-ability visas, also add tailored calibration text in `panel.ts`.

Add a new agent / AI step

Create `lib/agents/yourAgent.ts` that builds a prompt (stating the JSON shape) and calls `generateJSON`. Add a thin auth-gated `app/api/your-step/route.ts`. Call it from the client with `fetch`. Add validation/guardrails on the result.

Add a new page

Create `app/your-page/page.tsx` as a server component: `await auth()` guard, render `<AppHeader/>`, put interactivity in a sibling `"use client"` component. Add a nav link in `components/AppHeader.tsx`.

Add saved assessment history (next feature)

Add an `assessments` table in `lib/db.ts` `ensureSchema()` (`id`, `user_email`, `visa_id`, `result jsonb`, `created_at`). After a panel run, POST the result to a new route that inserts it. List them on `/assess`. The DB and auth are already wired.

11 Testing

There is no test framework — just standalone smoke scripts in `scripts/`, run with `npx tsx`. They need the relevant env var exported. Examples:

```
export $(grep '^GR0Q_API_KEY=' .env.local | xargs)
npx tsx scripts/test-panel.mts      # adversarial panel on sample evidence
npx tsx scripts/test-sources.mts    # GitHub + OpenAlex fetchers (no key needed)
npx tsx scripts/test-policy.mts     # Federal Register + classifier
DATABASE_URL="..." npx tsx scripts/test-db.mts  # profile round-trip

npx tsc --noEmit                    # typecheck the whole app (scripts/ is excluded)
```

When you add non-trivial logic, leave one runnable check behind in the same style. Typecheck before committing.

Caliber context handbook · give this to your AI assistant before editing the codebase. It reflects the state at the time of writing; when in doubt, read the actual files — the code is the source of truth.